NAME

compare − longest common subsequence of two files

SYNOPSIS

compare [*options*] *file1 file2*

DESCRIPTION

compare computes the Longest Common Subsequence (LCS) of *file1* and *file2*. By default it prints their differences as an edit script; with **-c** it prints the common (matching) records instead, and with **-l** it prints only the LCS length.

Each difference is introduced by a header line:

<<<<< n *file1* [*begin*:*end*] <<<<<
(records present only in *file1*)
>>>>> n *file2* [*begin*:*end*] >>>>>
(records present only in *file2*)

where *n* is the one-based difference number and *[begin:end]* the half-open range of records involved. A final line reports the LCS length, and when compiled with duration reporting enabled, a timing line such as *RickSpace: 28 operations in 9 µs @3.111 MHz* follows.

The two files are compared at one of three granularities:

record each input line is one record (the default);

text (**-T**) the whole file is one sequence of Unicode code points, so multi-byte characters are never split;

word (**-w**) each whitespace-separated word is one record.

Four LCS algorithms â or automatic dispatch â are available through **-a**:

a Automatic (the default): heuristic dispatch that selects the algorithm from the input's edit distance, match count, and alphabet size.

d Myers (1986): the O(ND) minimal edit-script algorithm, driven by the edit distance D; fastest on near-identical input.

m Hunt-Szymanski (1977): a threshold array over the match lists, O((r+n) log n) time.

s RickSpace (Rick 2000): forward/backward contours plus Hirschberg recursion, O(ns) space.

t RickTime (Rick 1995): a single diagonal sweep, O(ns + d) space.

Automatic dispatch selects Myers when the edit distance is small, Hunt-Szymanski when the matches are sparse, RickTime when its match recording fits in memory, and RickSpace otherwise.

OPTIONS

-a *a|d|m|s|t*
Select the LCS algorithm: **a** (Automatic, the default), **d** (Myers), **m** (Hunt-Szymanski), **s** (RickSpace), or **t** (RickTime). Accepts**-ad** or **-a d**.

-b Ignore whitespace: collapse each run of whitespace to a single space before comparing, and expand the reported spans back to the original positions.

-c Print the correspondence (the common, matching records) instead of the differences.

-E Print the host environment (locale, processor, and memory) and exit.

-f Treat the input files as FASTA and strip header lines; implies **-T**.

-h Print a usage synopsis and exit.

-i Perform a case-insensitive comparison.

-j *join* Coalesce nearby differences that lie within *join* records of each other.

-l Report only the LCS length; skip reconstructing the differences.

-p *prefix*
Print *prefix* lines of context before each difference.

-q Treat *file1* and *file2* as literal strings rather than filenames; implies **-T**.

-s *suffix*
Print *suffix* lines of context after each difference.

-t *tabsize*
Set the tab size used to expand tab characters in the parallel **-y** p output (default 8).

-T Compare the whole file as a sequence of Unicode code points (the text path) rather than line-by-line records.

-v Enable verbose output: print the LCS length, the start and finish wall-clock times, and a timing summary; by default only the differences are shown.

-V Print the version and exit.

-w Split each line into whitespace-separated words and compare word by word (record mode).

-W *width*
Set the per-side content width used by the parallel **-y** p format (default 80, minimum 5). Content is truncated to this width, so the total output line is 1 + 2 x width characters.

-y *s|p|c* Select the output format: **s** (sequential, the default: one side after the other), **p** (parallel: side-by-side columns), or **c** (CIGAR alignment string; file1 is the reference and file2 the query).

The **-h**, **-V**, and **-E** switches are early-exit switches: each prints its report and terminates without comparing files.

ALGORITHM

The **-a** selector chooses among four algorithms, each fastest in its own regime. Three independent properties of the input decide which regime applies:

Edit distance
D: insertions plus deletions of the minimal edit script.

Redundancy ρ
r/n: average matches per position; low for sparse matches, high for skewed or small alphabets.

Alphabet size
s: number of distinct symbols, which sizes the Rick table builds.

The algorithms and their regimes:

d Myers (edit distance): fastest on near-identical input (small *D*). The O(ND) minimal edit-script algorithm; its reconstruction history grows with the square of *D*, so it is bounded by a memory-derived budget.

m Hunt-Szymanski (matches): fastest when the matches are sparse (low ρ, large alphabet), the regime its threshold array is built for.

s RickSpace (space): the linear-memory fallback; O(ns) space via Hirschberg recursion, so it never exhausts memory and completes when no other algorithm does.

t RickTime (time): fastest at high redundancy (high ρ), when its dominant-match recording fits in memory.

Automatic dispatch (**-aa**, the default) estimates *D* and *r* from the symbol counts and selects in order:

1. Myers when the edit distance lower bound is at most 2000;
2. Hunt-Szymanski when the match count is at most 100 times the shorter input length (sparse matches);

3. RickTime when its recording budget covers the shorter input;
4. RickSpace otherwise.

-v reports which algorithm automatic dispatch selected and why.

EXIT STATUS

0 Success (including **-h**, **-V**, and **-E**).

1 Error: a command-line parse failure, a missing file, or an algorithm failure.

ENVIRONMENT

The locale (in particular **LANG**, **LC_ALL**, and **LC_NUMERIC**) controls the thousands separator used in the reported counts and the locale name shown by **-E**. On a system whose locale defines digit grouping, counts are printed with comma separators (for example **LCS Length = 4,591,960**); otherwise they are printed ungrouped.

EXAMPLES

Compare two files line by line and show the differences:

```
compare old.txt new.txt
```

Compare side by side (parallel output) with 40-character sides and 2-space tab stops:

```
compare -yp -W 40 -t 2 old.txt new.txt
```

Compare as text (code points), case-insensitively, showing only the length:

```
compare -i -l -T a.fa b.fa
```

Compare two FASTA genomes with the linear-space algorithm and show matches:

```
compare -as -c -f ecoli.seq ecoli_100subs.seq
```

Compare two literal strings rather than files:

```
compare -q "the quick sly fox jumped" "over the lazy brown dog"
```

SEE ALSO

diff(1), **cmp**(1)

The LCS algorithms and their performance bounds are described in *LCSRickSpace.md* and *LCSRick-Time.md* in the project documentation.